QMWS Spring 2026: Introduction to Python for Data Analysis

Day One: Foundations

Gavin Klorfine & Deborah Laze

Section 1

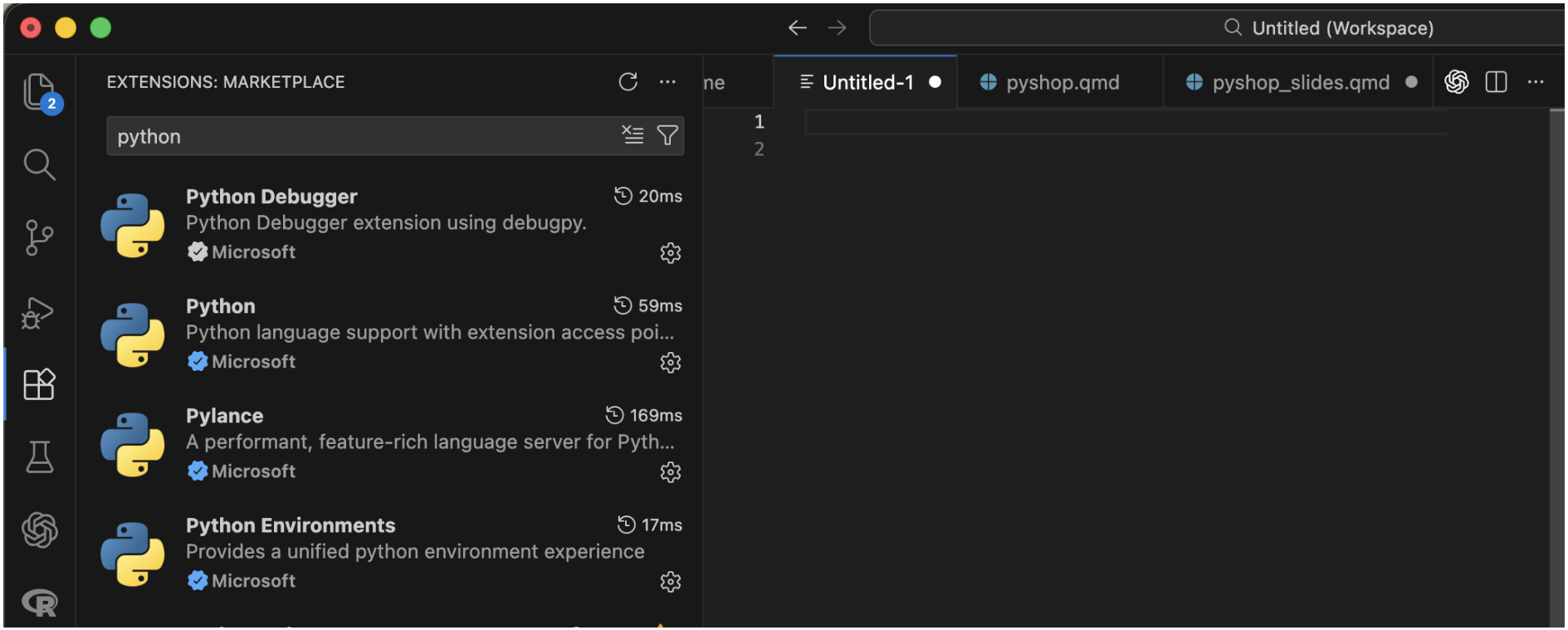
Installation

Visual Studio (VS) Code

- Installation link: https://code.visualstudio.com/download
- Integrated Development Environment (IDE)
- Can use it to code in many different languages
- Commonly used in both academia and industry

Visual Studio (VS) Code

- Once installed, go to the “Extensions” tab and search “Python”
- Install the below extensions:



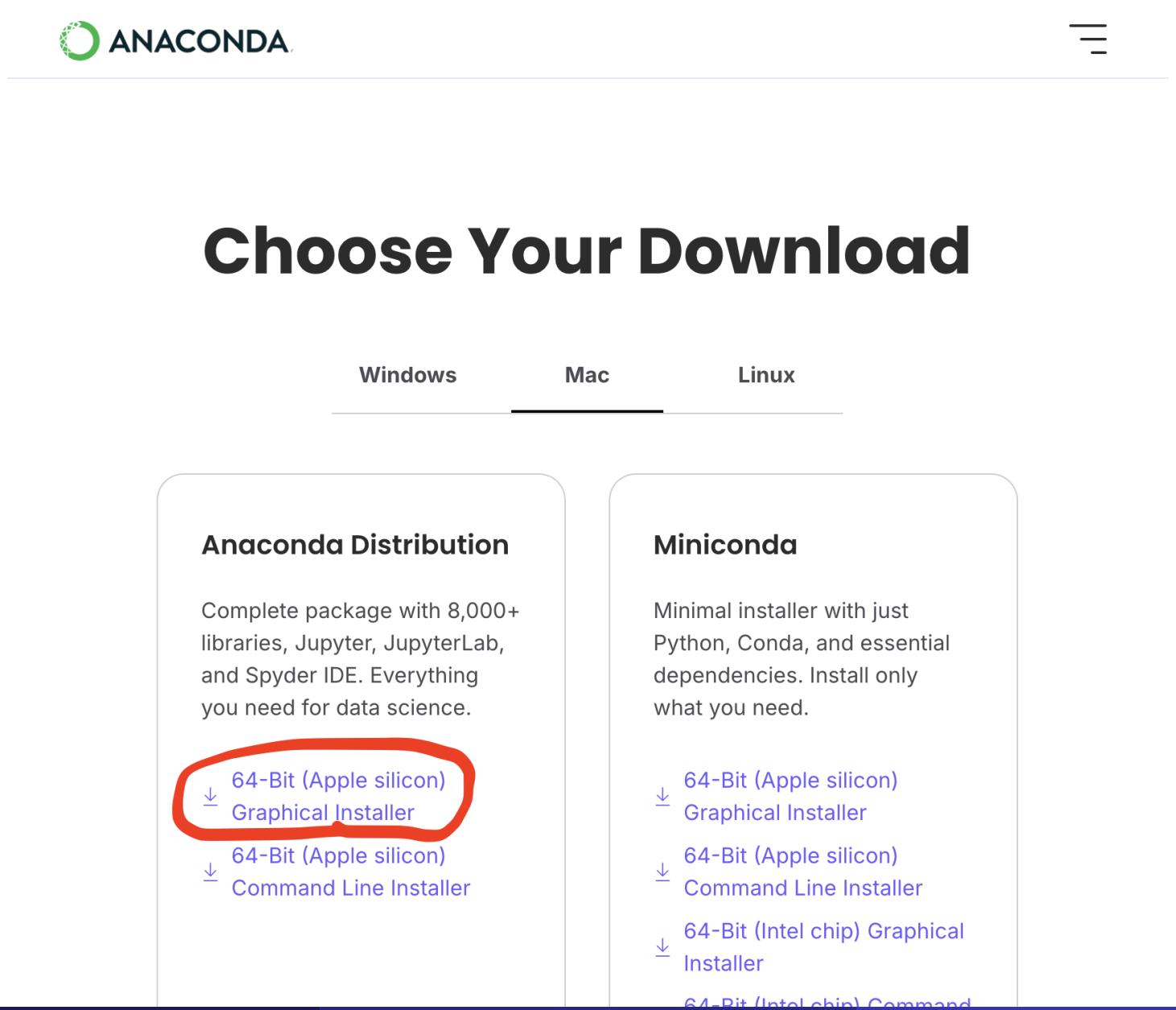
Anaconda

- A tool for configuring Python **environments**
  - More often than not, you need to install **packages** that allow you to code more easily/efficiently
  - Allows for version control
  - Issues can arise if environments are not used, for example:
    - If you install two packages that conflict with one another
    - If you install a package that requires a specific version of Python
  - Anaconda allows these issues to be isolated to a specific environment, rather than affecting your whole Python installation
    - Especially useful if you have more than one project going on

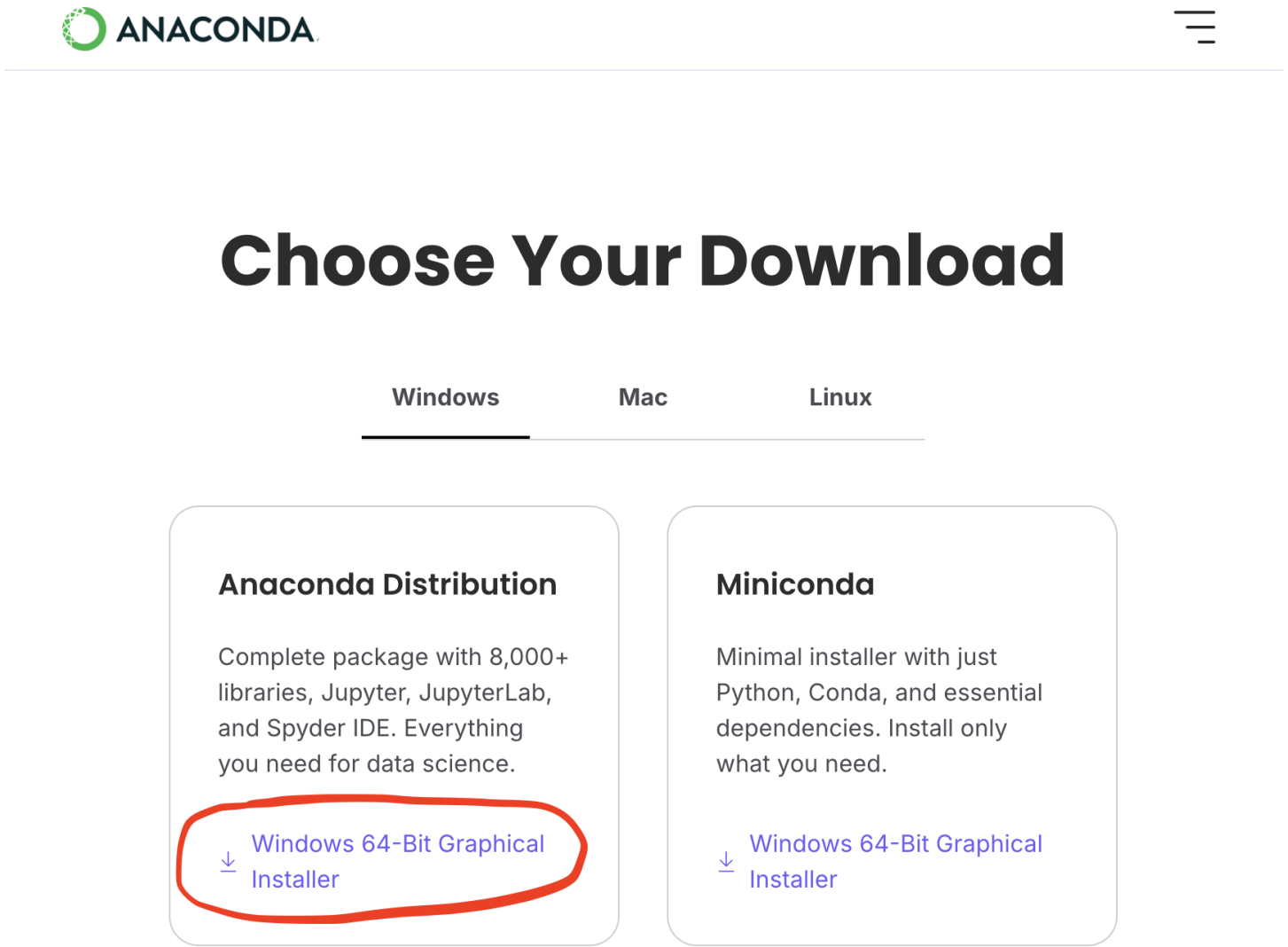
Anaconda

- Installation link: https://www.anaconda.com/download
- The simplest way to use Anaconda is via the **Navigator**
- To install:
  1. Make an account
  2. Install the full distribution via the "Graphical Installer"

- Installation link: https://www.anaconda.com/download
- On Mac:



- Installation link: https://www.anaconda.com/download
- On Windows:



Section 2

Setting Up a Conda Environment

Section 3

The Basics

Variables

- In programming, a variable is a space in computer memory where values may be stored

Defining and Printing Variables

```
x = 67

print(x)
```

```
67
```

Variables

Arithmetic

```
a = 1
b = 2

print(a + b)     # * for multiplication, / for division
```

```
3
```

Note

- Use # to make a **comment**

Variables

Updating Variables

```
x = 10
x = 20
x = 15

print (x)
```

```
15
```

Variable Types

1. `int` (integer; "whole number")
   - less memory
   - 1 is an `int`
2. `float` (real; "decimal number")
   - 1.2 is a `float`
   - 1.0 is a `float`
3. `str` (string; characters/words with no numerical meaning)
   - `"Hello world!"` is a string
   - `"416-676-6767"` is also a string
4. `bool` (boolean; can be `True` or `False`; more on this later)
5. `None`
   - A special type of variable that essentially means "nothing meaningful is stored"
   - Useful as a placeholder

Variable Types

Checking Type

```
x = 416
print(x, type(x))    # type() to check type
```

```
416 <class 'int'>
```

Converting Type

```
x = 416
x = str(x)       # str() to convert to string
print(x, type(x))
```

```
416 <class 'str'>
```

- Can also use `int()` and `float()` to convert to respective type

A Brief Note

- `print()`, `type()`, `str()`, etc. are all examples of **functions**
  - These are essentially pre-made chunks of code for you to use
  - More on this later!

What will the below code output?

```
x = "Variable"

print(type(x))
print(x)
```

Section 4

More Advanced Variable Types

Lists

- Multiple values can be stored in one variable. A list is one way to do this:

Defining Lists

```
x = [0, 1, 2, 3, 4]      # Define a list; square brackets
print(type(x))
```

```
<class 'list'>
```

Lists: Indexing

```
x = [0,1,2,3,4]
```

- To extract one or more values from a list, you **index** the list
- Python uses **zero-based indexing**
  - This means that the first element of a list is index `0`, the second element is index `1`, ...
  - Element x is index $x - 1$

Lists: Indexing

Indexing Lists

```
x = [0, 1, 2, 3, 4]
y = 2

print(x[0])      # Index first element of a list
print(x[-1])     # Index last element
print(x[y])      # Index yth element
```

```
0
4
2
```

Lists: Indexing

- You can also take more than one element, or a **slice**, of the list

Slicing Lists

```
x = [0, 1, 2, 3, 4]

print(x[0:2])   # first and second (but not third) elements
```

```
[0, 1]
```

Important

- `[inclusive:exclusive]`
  - The index on the *left-hand* side of the slice is *included*, whereas the index on the *right-hand* side of the slice is *excluded*

Lists: Indexing

Slicing Lists, continued

```
x = [0, 1, 2, 3, 4]

print(x[0:3])   # first three elements
print(x[0:1])   # a (silly) way to index just the first elemer
```

```
[0, 1, 2]
[0]
```

Lists: Length

- To find the length of a list:

len() function

```
x = [0, 1, 2, 3, 4]

print(len(x))   # The length of list x

# Another way to find the last element (not so silly!)
print(x[len(x) - 1])
```

```
5
4
```

Lists: Methods

- Lists have certain functions that can be applied to them, called **methods**
  - Methods are more general than this (e.g., string methods; more on these in a little bit!)

.append()

```
x = [0,1,2,3,4]

x.append(5)
print(x)
```

```
[0, 1, 2, 3, 4, 5]
```

- Adds an entry to a list

Lists: Methods

.extend()

```
x = [0,1,2,3,4]

x.extend([6,7,8,9])
print(x)
```

```
[0, 1, 2, 3, 4, 6, 7, 8, 9]
```

- Extend a list by "glueing" it to another list

Lists: Methods

Difference Between .append() and .extend()

```
x = [0,1,2,4]
y = [5,6,7,8]

x.append(y)
print(x)

x = [0,1,2,4]    # Redefine x
x.extend(y)
print(x)
```

```
[0, 1, 2, 4, [5, 6, 7, 8]]
[0, 1, 2, 4, 5, 6, 7, 8]
```

Lists: Changing Values

Changing a Value in a List

```
x = [0, 1, 2, 3, 4]

x[0] = 9
print(x)
```

```
[9, 1, 2, 3, 4]
```

Tuples (Briefly)

- Like lists, tuples contain multiple values
- Unlike lists, tuples are **immutable**; they cannot be changed once created.
  - Lists are *mutable*

Defining a Tuple

```
x = ("a", "tuple")  # Round brackets

print(x[0], x[1])
print(type(x))
```

```
a tuple
<class 'tuple'>
```

Tuples (Briefly)

Changing Values... ERROR!

```
x = ("a", "tuple")

x[0] = "Not a"
```

```
TypeError: 'tuple' object does not support item assignment
---------------------------------------------------------------
TypeError                                 Traceback (most rece
Cell In[345], line 3
      1 x = ("a", "tuple")
----> 3 x[0] = "Not a"
TypeError: 'tuple' object does not support item assignment
```

Tuples (Briefly)

- Tuples have special properties, one of which is below:

Unpacking Tuples

```
x, y = (6, 7)

print(x)
print(y)
```

```
6
7
```

Section 5

Strings

Strings

- Strings are a lot like lists in that they are **iterable**
  - For now, think of these (lists, strings) as a sequence of values

Strings are Like Lists!

```
cringe = "Hello world!"

print(cringe[0:5])
print(len(cringe))
```

```
Hello
12
```

Strings

- We can join two strings together through **concatenation**

Concatenation

```
a = "Hello"
b = "World!"

c = a + " " + b      # Note the space between words
print(c)
```

```
Hello World!
```

Strings

- Sometimes you need to include quotations or other characters inside a string that would normally return an error
- This can be done through an **escape sequence**

ERROR! Why You Need Escape Sequences...

```
x = "This is a "string""
```

```
SyntaxError: invalid syntax (886282512.py, line 1)
  Cell In[349], line 1
    x = "This is a "string""
                   ^
SyntaxError: invalid syntax
```

Strings

Escape Sequences

```
x = "This is a \"string\""
print(x)
```

```
This is a "string"
```

Note

- The

Strings

- Strings are also kind of like tuples in that they are **immutable**

String Immutability

```
cringe = "Hello world!"
cringe[0:5] = "Goodbye"      # Error
```

```
TypeError: 'str' object does not support item assignment
---------------------------------------------------------------
TypeError                                 Traceback (most rece
Cell In[351], line 2
      1 cringe = "Hello world!"
----> 2 cringe[0:5] = "Goodbye"      # Error
TypeError: 'str' object does not support item assignment
```

Strings

- We can turn to string **methods** to "modify" strings when needed (they just create a new string "behind the scenes")

.replace()

```
cringe = "Hello world!"

print(cringe.replace("Hello", "Goodbye"))
```

```
Goodbye world!
```

Strings

.split()

```
meme_list = "baby gronk, 67, skibidi toilet, the rizzler"


print(meme_list.split(", ")) # Split on ", "; returns a list

print(meme_list.split(", ")[0]) # Index the list
```

```
['baby gronk', '67', 'skibidi toilet', 'the rizzler']
baby gronk
```

Strings

How might you index “the rizzler”?

Strings

How might you index “the rizzler”?

```
meme_list = "baby gronk, 67, skibidi toilet, the rizzler"

print(meme_list.split(", ")[3])
print(meme_list.split(", ")[-1])
print(meme_list.split(", ")[len(meme_list.split(", ")) - 1])
```

```
the rizzler
the rizzler
the rizzler
```

Strings

- If you wanted to include variables within text, format strings as per the following:

f-string Example

```
x = 3.14159

print(f'Approximate value of pi is {x}')

print(f'To three decimal places is {x:.3f}')
```

```
Approximate value of pi is 3.14159
To three decimal places is 3.142
```

Strings

- The string (surrounded by either single `''` or double `""` quotations) is preceded by `f`
- The variable is surrounded by curly braces `{}` within the string
- Formatting is applied to the variable *within the curly braces*
  - A `:` signals that the proceeding formatting code should be applied to the variable

f-string Formatting Basics

- `x:.3f`
  - Format variable `x`
  - Round to 3 decimal places
  - `f` means `float`

Section 6

Booleans

Booleans

- A boolean variable (type `bool`) can be either `True` (`1`) or `False` (`0`).

Booleans

```
x = True
y = 10
print(type(x), type(y))

print (y > 5)
print (y < 5)
print(type(y < 5))
```

```
<class 'bool'> <class 'int'>
True
False
<class 'bool'>
```

Booleans

Comparison Operators

- > greater than
- < less than
- >= greater than or equal to
- <= less than or equal to
- == equal to
- != not equal to

Logical Operators

- `and`
- `or`
- `not`

Section 7

Conditional Statements

Conditional Statements

- If thing1 happens, do x
- If thing2 happens instead, do y
- Otherwise, do z

Conditional Statements

- If thing1 happens, do x
- If thing2 happens instead, do y
- Otherwise, do z

Basic `if`/`elif`/`else`

```
x = 10  # Value presumably unknown
y = 20

if x == 4:
    print("x is 4")
elif x > y:
    print("x is bigger than y")
else:
    print("none of the above")
```

```
none of the above
```

Conditional Statements

Structure of Conditionals

```
if x == 4:
    print("x is 4")
```

- Begins with `if`/`elif` ("else if")/`else`
- Boolean expression/value
- Colon :
- Indented code underneath that runs if condition is met

Section 8

Loops

`while` Loops

- Loops are used to run a chunk of code repeatedly
  - Often a set number of times
- This is best shown and unpacked through examples:

while Loops

while Loop Basics

```
count = 1    # Initialize counter

while count <= 5:
    print(count)
    count = count + 1    # Increment counter
```

```
1
2
3
4
5
```

while Loops

Iterating through a list

```
x = [0,1,2,3]
count = 0

while count <= len(x) - 1:
    print(x[count])
    count += 1    # Same as count = count + 1
```

```
0
1
2
3
```

while Loops

- The `break` statement is used to exit a loop

break

```
x = 0

while True:      # Infinite loop...
    if x > 10:
        break    # Until you break!
    x += 1

print(x)
```

```
11
```

for Loops

- Often used with the `range()` function to loop over code a set number of times

for Loops, range()

```
for i in range(3):
    print(i)
```

```
0
1
2
```

for Loops

Iterating Over Entries in a List

```
x = [10,20,30,67]

for i in x:
    print(i)
```

```
10
20
30
67
```

for Loops

- Use `enumerate()` and a second indexing variable (usually `i` or `j`) if you want to keep variable position

enumerate()

```
x = [10,20,30,67]

for i, j in enumerate(x):
    print(i, j)
```

```
0 10
1 20
2 30
3 67
```

for Loops

- Loops (and conditionals) may be nested inside one another (like Russian dolls / Matryoshka)

Nesting

```
x = ["big", "small"]
y = ["cat", "dog"]

for i in x:
    for j in y:
        print(i, j)
```

```
big cat
big dog
small cat
small dog
```

Section 9

Importing Packages, Defining Functions

Importing Packages

- Some packages are installed with Python by default, and no extra work is required to use them

import math

```
import math

print(f'pi = {math.pi:.3f}')
```

```
pi = 3.142
```

Importing Packages

- We can rename packages to make their usage more concise

Renaming Packages

```
import math as m

print(f'pi = {m.pi:.3f}')
```

```
pi = 3.142
```

Importing Packages

- Some require installation. This is most easily (and safely) done through **anaconda**

Importing Packages

import numpy as np

```
import numpy as np       # Common convention

# Take a sample of 3 values from N(0, 1)
print(np.random.normal(loc = 0, scale = 1, size = 3))
```

```
[-0.97839062 -0.42252989  0.02387375]
```

Note

- loc $= \mu$, scale $= \sigma$

Defining Functions

- Sometimes you repeat the same code over and over again
  - When this happens, it is useful to collapse the repeated code into a function
  - Otherwise, code becomes long and messy due to copy-pasting

Defining Functions

Defining Basic Functions

```
def addition(a, b):
    return a + b                    # What the functions spits out

print(addition(a = 1, b = 5)) # Call function

def subtraction(a, b = 1):      # Give b default value of 1
    return a - b

print(subtraction(a = 5))
```

```
6
4
```

Defining Functions

Function Example

```
x = [0,1,2,3,4,5,6,7]

def qm_mean(things):
    sum = 0

    for i in things:
        sum += i

    return sum / len(x)

qm_mean(x)
```

```
3.5
```

Section 10

Numpy

Numpy

- Somewhat similar to MATLAB if you have used that
- Useful for working with arrays/matrices
- Stands for **num**erical **py**thon
- Based in computer language **C**
  - Faster, more efficient computations

Numpy

Working With Lists/Arrays

```
import numpy as np

list1 = [1,2,3,4]
print(list1 * 2)
print(type(list1))

numpyList = np.array(list1)  # Convert list -> numpy array
print(numpyList * 2)
print(type(numpyList))
```

```
[1, 2, 3, 4, 1, 2, 3, 4]
<class 'list'>
[2 4 6 8]
<class 'numpy.ndarray'>
```

Numpy

- Matrices are typically worked with as **2D** numpy arrays

2D Numpy Arrays

```
import numpy as np

array2d = np.array([[1,2,3], [4,5,6]]) # List of lists
print(array2d)
print(type(array2d))
```

```
[[1 2 3]
 [4 5 6]]
<class 'numpy.ndarray'>
```

Numpy:

To Check Attributes of a Numpy Variable

```
import numpy as np

array2d = np.array([[1,2,3], [4,5,6]])

print(array2d.shape)     # Shape (# rows, # columns)
print(array2d.ndim)      # Number of dimensions
print(array2d.size)      # Number of elements
print(array2d.dtype)     # Type of elements
```

```
(2, 3)
2
6
int64
```

Numpy

- Some numpy types:
  - `uint8`
  - `int32`
  - `int64`
  - `float64`
- Generally, numpy type = type learned before + memory (in bits)

Numpy

np.arange()

```
import numpy as np

np.arange(start=10, stop=20, step=2, dtype = "int64")
```

```
array([10, 12, 14, 16, 18])
```

Note

- stop parameter is *exclusive*

Numpy

np.linspace()

```
import numpy as np

np.linspace(start=10, stop=20, num=6, dtype = "int64")
```

```
array([10, 12, 14, 16, 18, 20])
```

Tip

- Use `np.linspace()` for breaking something into equal parts
- Use `np.arange()` for creating a sequence of numbers

Numpy

Random Numbers

```
import numpy as np

print(np.random.normal(loc = 0, scale = 1, size = 3))
print(np.random.uniform(low = 0, high = 10, size = 3))
```

```
[-0.04473725  1.57446849  0.68187257]
[1.67658619 8.78555258 5.43946075]
```

Numpy

Indexing Numpy Arrays

```
import numpy as np

x = np.random.normal(0, 10, size=(3,3))
print(x, "\n")  # Print x with a blank line after it

print(x[2,2], type(x[2,2]))
print(x[2,:])   # Row 2 (zero-based), all columns
```

```
[[ -7.14541558   2.37751815 -19.04609116]
 [ 15.68967258 -18.70464672 -13.59293862]
 [ -7.52555632  -3.04316705  -2.06629967]]

-2.0662996677879932 <class 'numpy.float64'>
[-7.52555632 -3.04316705 -2.06629967]
```

Numpy

Indexing (cont.)

```
import numpy as np

x = np.random.normal(0, 10, size=(3,3))
print(x, "\n")

print(x[2, 0:2])    # Row 2, columns 0 and 1
print(type(x[2, 0:2]))
```

```
[[ 3.26642388  0.53753934 18.04552821]
 [ 5.50713435 15.18511963 -9.95748343]
 [ 3.02246923  8.64316482  1.96504596]]

[3.02246923 8.64316482]
<class 'numpy.ndarray'>
```

Numpy

- You may also perform **aggregate** functions on numpy arrays

Numpy Aggregate Functions

```
import numpy as np

x = np.array([1,4,7,15])

print(np.mean(x))
print(np.median(x))
print(np.std(x))
```

```
6.75
5.5
5.2141634036535525
```

Numpy

More Aggregate Functions

```
import numpy as np

x = np.array([1,4,7,15])

print(np.min(x))          # Minimum value
print(np.argmin(x))       # Position of minimum value
print(np.max(x))          # Maximum value
print(np.argmax(x))       # Position of maximum value
```

```
1
0
15
3
```

Numpy

Check-in

- How might you generate 100 random values from the standard normal distribution?
- How might you get the maximum of these values?

Numpy

Check-in

- How might you generate 100 random values from the standard normal distribution?
- How might you get the maximum of these values?

```
import numpy as np

print(np.max(np.random.normal(loc=0, scale=1, size=100)))
```

```
2.591863121012858
```

Section 11

Pandas